LIN7076 – Foundations of Computational Linguistics

Vector Semantics

Adèle Hénot-Mortier

26/10/2025

Queen Mary University of London

Choice of features/independent variables

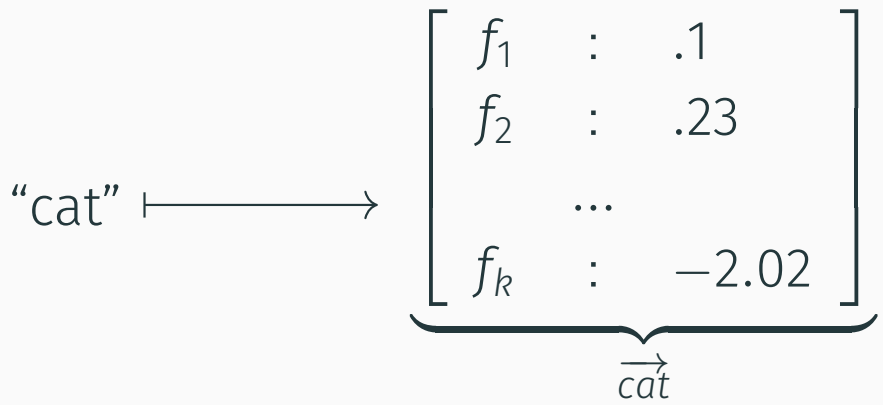
- So far we've seen methods requiring us to **pre-define certain features of interest** to make predictions:
  - with Naive Bayes, we have represented a piece of text as a **bag of words**;
  - with regression models, we have talked about variables like **speech rate**, **gender**, **age**, **socio-economic status**, **context**...
  - the regression labs also used features, like sentence length, average word length, percentage of real words, average word frequency (to predict child speaker age), or formants (to predict vowel quality).

Bypassing feature engineering

- So far we've seen methods requiring us to **pre-define certain features of interest** to make predictions.
- Ones needs to carefully pick these features, in a way that **does not overlook important factors**, but also **does not include redundant factors**... That's **feature engineering**, and may require a lot of trial-and-error.
- Additionally, one needs to **encode and label these features** to produce training data.
- It'd be great if we did not always have to hand-pick and model these features!

Featurizing words

- Today we'll see how to automatically **map words to a bunch of continuous features**, that we'll package together as vectors, called **word-vectors**.



- Models achieving this are called **word embedding models**, or simply embeddings.

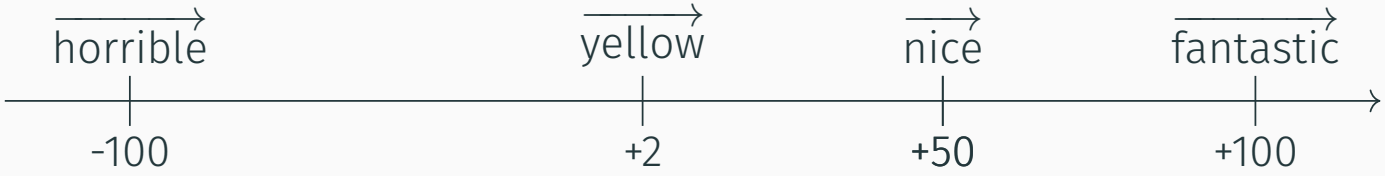
Advantages

- From a practical perspective, embeddings spare us some tedious feature engineering! And we can use word-vectors directly for **downstream tasks**, e.g. classification, regression.
- From a conceptual perspective, embeddings can be shown to capture **semantic and syntactic regularities**, e.g. valence (positive/negative sentiment), or syntactic category.
- But we'll also discuss some of their limits, focusing on adjectives.
- In some of my own research, I've been trying to show how these methods and their limits **inform us about our own linguistic competence**: are we purely statistical machines, or do we actively use real word information to learn words?

Modeling adjectives

1D Adjectives

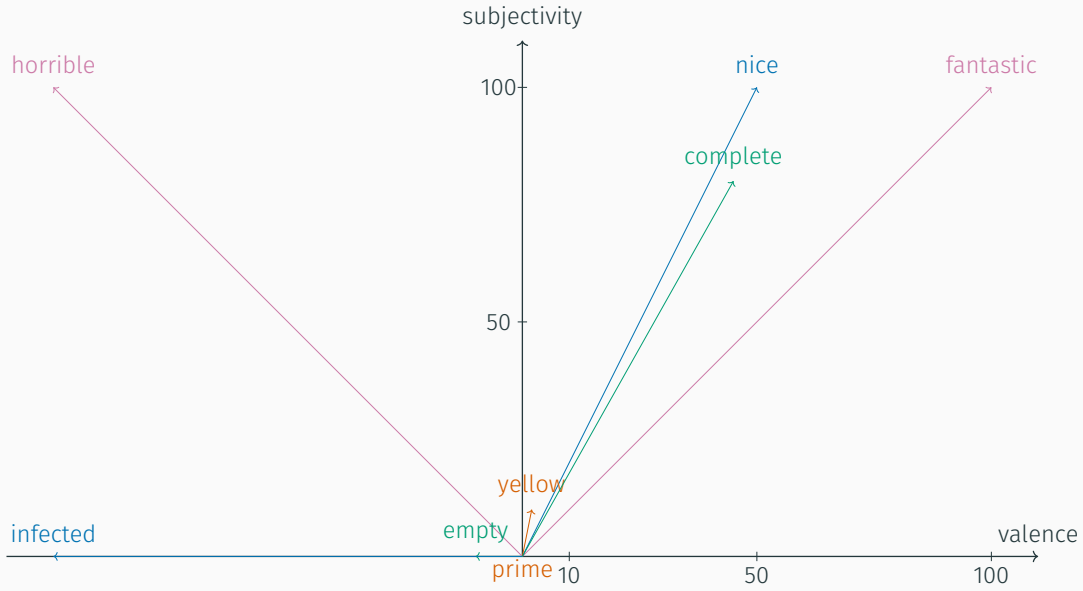
- Suppose we have a model mapping any adjective to 1 continuous variable. What could it be? For instance, **valence** (positive vs. negative sentiment).
- We have $\overrightarrow{\text{fantastic}} = 100$, $\overrightarrow{\text{horrible}} = -100$, $\overrightarrow{\text{nice}} = 50$, $\overrightarrow{\text{yellow}} = 2$.
- We can see these values as **vectors of dimension 1**, and represent them on a single axis.



- Can you think of other interesting features differentiating adjectives?

2D adjectives

- Suppose now we have a model mapping any adjective to 2 continuous variables: **valence** and **subjectivity**.
- For instance, we'd have $\overrightarrow{\text{fantastic}}$ = [100, 100], $\overrightarrow{\text{horrible}}$ = [-100, 100], $\overrightarrow{\text{nice}}$ = [50, 100], $\overrightarrow{\text{yellow}}$ = [2,10], $\overrightarrow{\text{empty}}$ = [-10,0], $\overrightarrow{\text{prime}}$ = [0,0], $\overrightarrow{\text{infected}}$ = [-100,0], $\overrightarrow{\text{complete}}$ = [45, 80]. What about *tasty*? *Tough*?
- We can see these values as **vectors of dimension 2**, and represent them on a 2D scatter plot, as arrows starting at the origin, or simply as the endpoints of these arrows.



Antonymy and morphological complexity

- Some adjectives like *tall* and *short* denote **opposite domains on the same scale**. They are **antonyms**.
- Even if antonyms are sematic opposites, they are measuring the same thing; so in some respects, *tall* and *short* are closer to each other than *tall* and *fit*.
- Additionally antonymic pairs display a **morphological asymmetry**: the "negative" adjective of the pair is much more likely than its "positive" counterpart to display overt negative morphology.

(1) a. Friendly-Unfriendly
    b. *Unmean-Mean

(2) a. Happy-Unhappy
    b. *Unsad-Sad

Gradability, context dependence and vagueness

- Some adjectives like *tall* have degrees, they are called **gradable**.
- Others like *dead* or *married*, less so. Diagnostics for gradability:
  - Measure phrase: *be 6ft tall* vs. *#be 2 years dead*.
  - Intensification: *very tall*, *??very dead*.
  - Comparison: *taller than*, *??more dead/deader than*
- Gradability is independent from valence, but somehow related to antonymy: antonyms will display same gradability.
- Gradable adjectives are very often **context-dependent**. A gigantic cockroach is still smaller than a juvenile T. Rex. The threshold (e.g. size in cm) for considering that the adjective applies, depends on which object it applies to! Some potential exception are end-of-scale adjectives like *full*.
- They are also often **vague**: there are grey cases for which not everybody will agree on whether the adjective can apply. That can be the case even for a fixed object, in a fixed context.

Stage vs. individual level

- Some adjectives like *French*, *smart*, *blue-eyed* signal **permanent**, stable properties. We call them **individual-level**. This class contains both gradable (e.g. *smart*) and non-gradable (e.g. *French*) adjectives!
- Some other adjectives like *sick*, *happy*, *naked*, are more **temporary**, reversible. We call them **stage-level**. This class contains both gradable (e.g. *happy*) and non-gradable (e.g. *naked*) adjectives!
- This division is vastly independent from valence and gradability.

Type of modification

- Many adjectives can be understood as the set of the elements of which the adjective is true. So *tall* is the set of tall things, *French* the set of French things etc.
- Many such adjectives are **intersective**, which means that their meaning and the meaning of their argument intersect. A *red car* is the set of things that are both *red*, and *cars*.
- But what about adjectives like *talented*? The meaning of *talented* depends on the meaning of its argument: a *talented chef* and a *talented linguist* do not have the same skills! Such adjectives further specify a subset of their argument: *talented chefs* are a subset of all the chefs. We call them **subsective**.
- What about *fake* or *alleged*? these are even weirder, because they change their argument into something different (a *fake gun* is not a *gun*!), or *potentially* different (an *alleged criminal* may or may not be a criminal). The former kind we call **privative**, the latter **non-subsective modal**.

Desiderata for an adjectival embedding

- We want a model *deriving* meaningful features that look like valence, antonymy/complexity, gradability, stage/individual-level, type of modification etc.

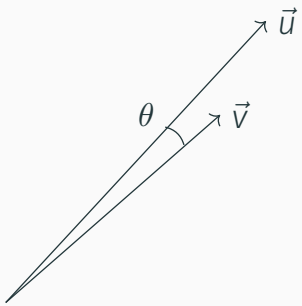
$$\text{“happy”} \longmapsto \underbrace{\begin{bmatrix} \textit{valence} & : & .8 \\ \textit{gradability} & : & .9 \\ \textit{stage} & : & .9 \\ \textit{intersective} & : & .9 \\ \textit{subsective} & : & .1 \\ \textit{privative} : & : & 0 \\ \textit{complexity} & : & 0 \end{bmatrix}}_{\overrightarrow{\textit{happy}}}$$

- Intuition: such features are derived by forcing the model to **assign semantically close words semantically close vectors**.
- What does it mean for two vectors to be close?

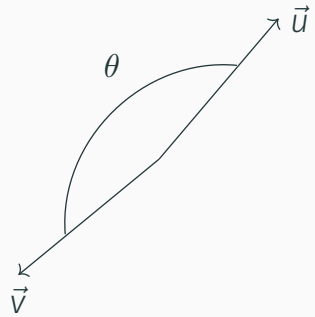
Meaningful relations in word embeddings

Vector similarity

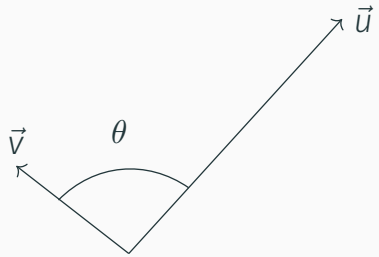
- A standard measure of similarity between word vectors is **cosine similarity** (*CosSim*):[1] it is a **function of the angle formed by two vectors**.
  - If the two vectors have same orientation and direction, their *CosSim* is 1, which implies the 2 word-vectors are very **similar**.
  - If they are orthogonal (i.e. form a 90-degrees angle in 2D), their *CosSim* is 0 which implies the 2 word-vectors are **unrelated**.
  - If they have same orientations but opposite directions, their *CosSim* is -1. This rarely happens in practice.



$CosSim(\vec{u}, \vec{v}) \sim 1$



$CosSim(\vec{u}, \vec{v}) \sim -1$



$CosSim(\vec{u}, \vec{v}) \sim 0$

[1] $CosSim(\vec{u}, \vec{v}) = \frac{\vec{u}.\vec{v}}{||\vec{u}|| \times ||\vec{v}||}$ with $\vec{u}.\vec{v}$ the dot product between $\vec{u}$ and $\vec{v}$ and $||\vec{u}|| \times ||\vec{v}||$ the product of their magnitudes (lengths).

CosSim computation practice 1 (optional)

- Let $\vec{u} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ and $\vec{v} = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}$.
- We compute the dot product of $\vec{u}$ and $\vec{v}$ by computing the sum of the products of each of their components:
  $\vec{u}.\vec{v} = 1 \times 2 + 0 \times 0 + 0 \times 0 = 2$.
- We compute the norms (lengths) of the vectors by taking the square root of the sum of their squared components:[2]
  $||\vec{u}|| = \sqrt{1^2 + 0^2 + 0^2} = 1$ and $||\vec{v}|| = \sqrt{2^2 + 0^2 + 0^2} = 2$.
- We divide the dot product by the product of the norms to get the *CosSim*: $CosSim(\vec{u}, \vec{v}) = \frac{2}{1 \times 2} = 1$.
- Adds up: $\vec{u}$ and $\vec{v}$ have same direction: that of their first component.

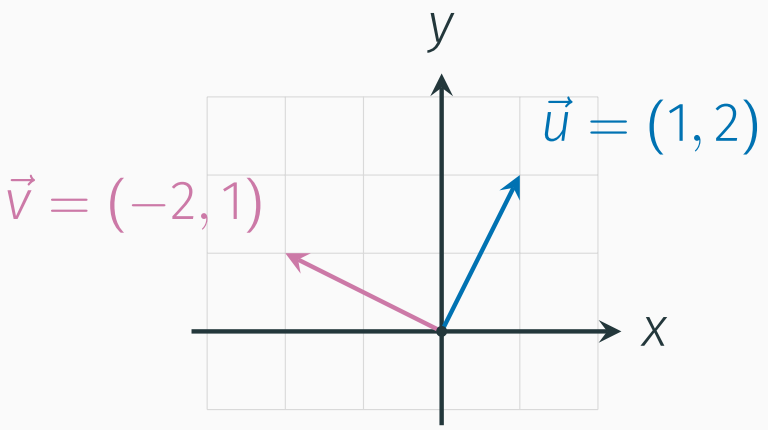
[2] You can see that as a generalization of the Pythagorean theorem to more than 2 dimensions, $a^2 = b^2 + c^2 \iff a = \sqrt{b^2 + c^2}$.

CosSim computation practice 2 (optional)

- Let $\vec{u} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ and $\vec{v} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}$.
- We compute the dot product of $\vec{u}$ and $\vec{v}$ by computing the sum of the products of each of their components:
  $\vec{u}.\vec{v} = 1 \times 2 + 0 \times 1 + 0 \times 0 = 2$. Same as before.
- We compute the norms (lengths) of the vectors by taking the square root of the sum of their squared components:
  $||\vec{u}|| = \sqrt{1^2 + 0^2 + 0^2} = 1$ (same as before) and
  $||\vec{v}|| = \sqrt{2^2 + 1^2 + 0^2} = \sqrt{5} = \simeq 2.24$.
- We divide the dot product by the product of the norms to get the *CosSim*: $CosSim(\vec{u}, \vec{v}) \simeq \frac{2}{1 \times 2.24} = .89$.
- Adds up: $\vec{u}$ and $\vec{v}$ have similar but different directions: $\vec{u}$ is following the first component, while $\vec{v}$ is following a mix of the first and second components.

CosSim computation practice 3 (optional)

- Let $\vec{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ and $\vec{v} = \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$.
- We compute the dot product of $\vec{u}$ and $\vec{v}$ by computing the sum of the products of each of their components: $\vec{u}.\vec{v} = 1 \times (-2) + 2 \times 1 + 3 \times 0 = -2 + 2 = 0$. Because this is zero, we can be sure the *CosSim* of $\vec{u}$ and $\vec{v}$ is zero, too. No need to bother computing the norms.

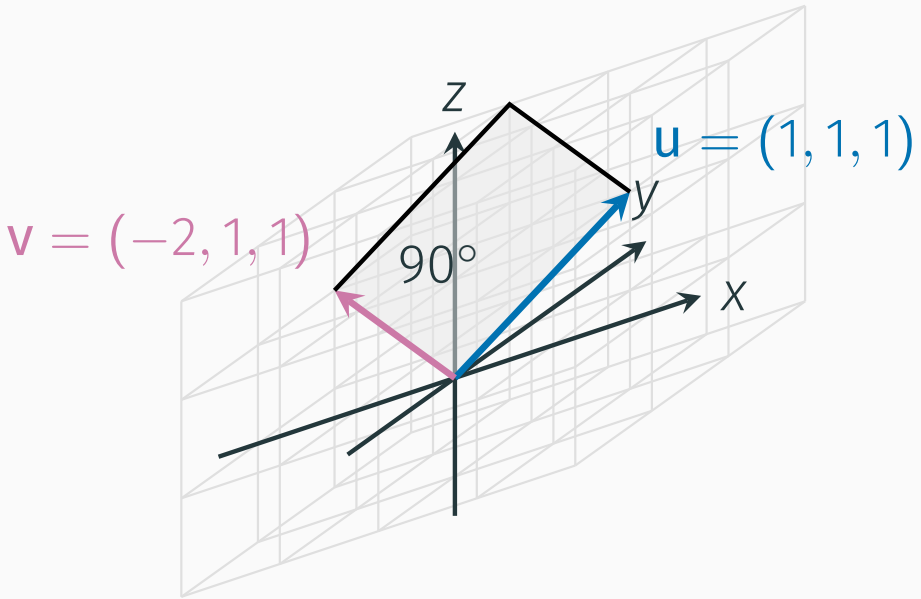


Made with Chat-GPT TikZ code.

- A *CosSim* of 0 makes sense geometrically: $\vec{u}$ and $\vec{v}$ form a right angle when projected on the plane formed by their first 2 components.

CosSim computation practice 4 (optional)

- Let $\vec{u} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ and $\vec{v} = \begin{bmatrix} -2 \\ 1 \\ 1 \end{bmatrix}$.
- We compute the dot product of $\vec{u}$ and $\vec{v}$ by computing the sum of the products of each of their components: $\vec{u}.\vec{v} = 1 \times (-2) + 1 \times 1 + 1 \times 1 = -2 + 1 + 1 = 0$. Because this is zero, we can be sure the *CosSim* of $\vec{u}$ and $\vec{v}$ is zero, too. No need to bother computing the norms.



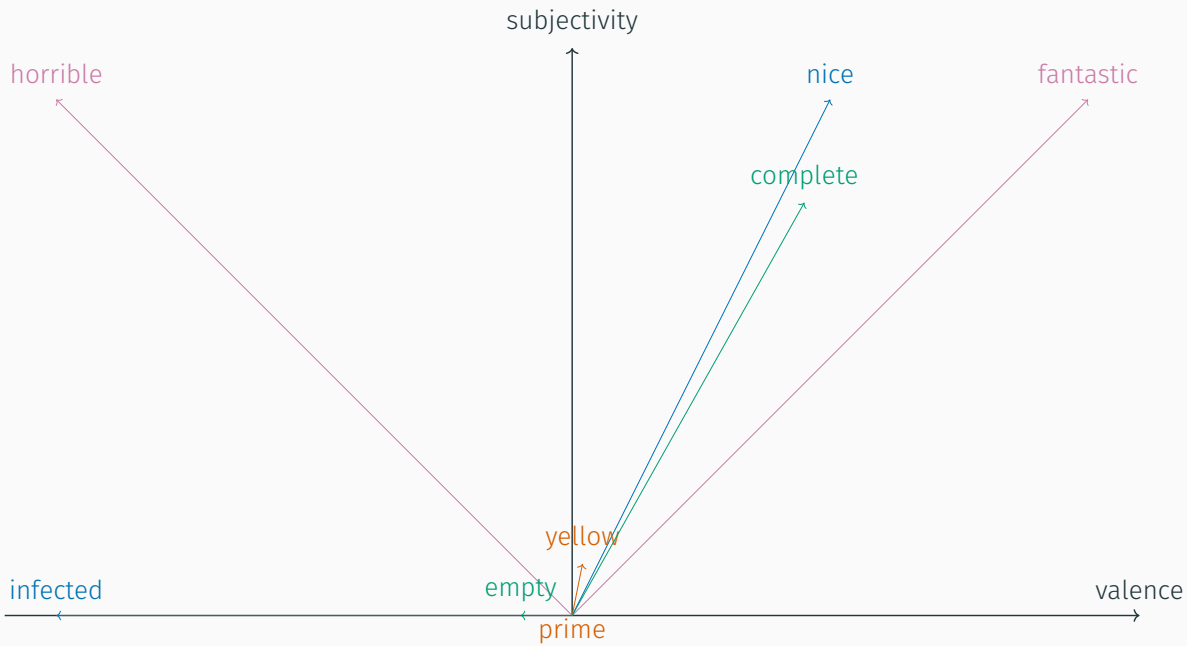
- A *CosSim* of 0 makes sense geometrically: $\vec{u}$ and $\vec{v}$ form a right angle when projected on the plane they define.

Made with Chat-GPT TikZ code.

Cosine similarity as a “similarity in recipe”

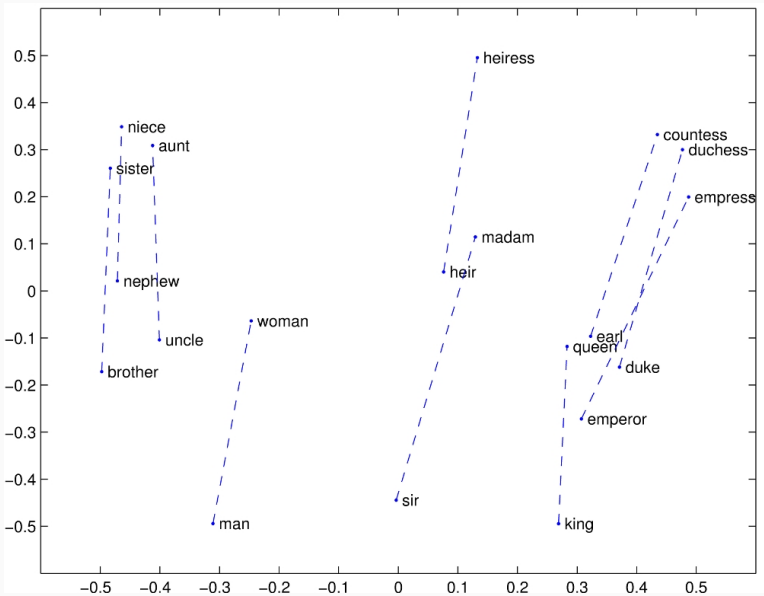
- Two vectors will be very similar in terms of *CosSim* if their various components (features) are in the same **proportions**. The lengths (magnitudes) of the two vectors are not taken into account.
- Two cosine-similar vectors will be **using the same “recipe”**, in which their features are the ingredients and their values their quantities. One vector may be cooking for 10 people and the other may be cooking for 50 people, but the recipe is the same.
- Two vectors will be unrelated if they use completely different ingredients, i.e. they cook different dishes.

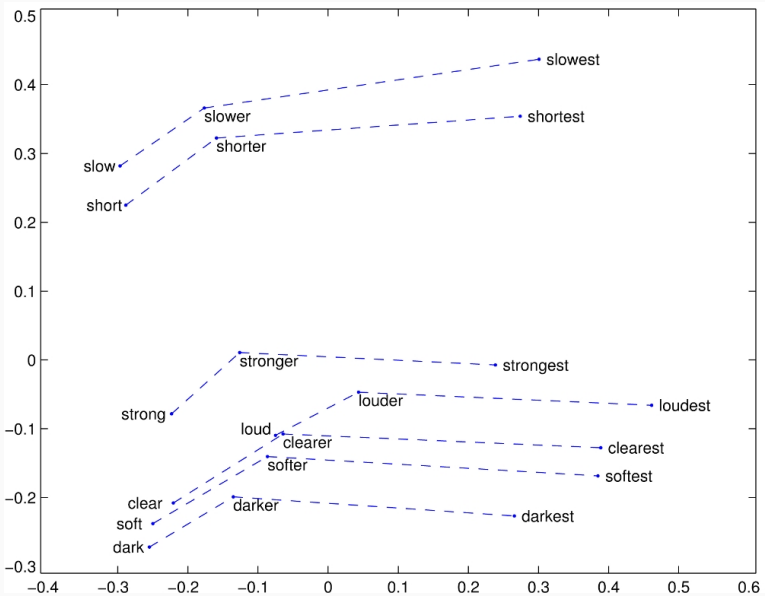
Vector similarity practice



- Which vector is closer to $\overrightarrow{\text{fantastic}}$ in terms of *CosSim*, $\overrightarrow{\text{complete}}$ or $\overrightarrow{\text{nice}}$?
- Vector(s) farthest from $\overrightarrow{\text{fantastic}}$?
- Vectors(s) unrelated to (i.e. orthogonal with) $\overrightarrow{\text{fantastic}}$?
- What is not amazing about this vector space?

Other cool stuff with word vectors: analogies

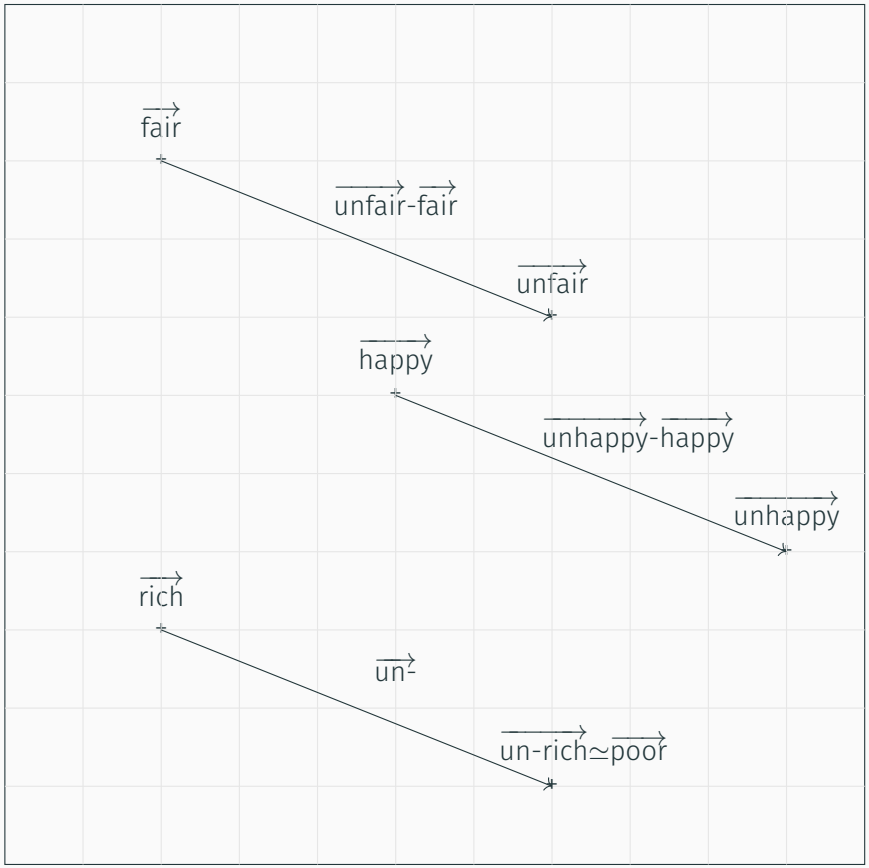




- If word-vectors are meaningful, then **the vectors linking pairs of word-vectors can also be meaningful**.
- Vectors from *brother* to *sister*, from *king* to *queen*, from *man* to *woman*, appear similar: same direction, orientation, and length.
- Vectors from positive-form (e.g. *strong*) to comparative (*stronger*), to superlative (*strongest*) adjectives are also very stable!

Analogy practice

- Where should $\overrightarrow{\text{poor}}$ be? We assume *poor* means *un-rich*.



- Step 1: determine the vector for *-un*: [5, -2].[3]
- Step 2: add this vector to $\overrightarrow{\text{rich}}$.
- Step 3 (optional): we can look at the words whose vectors are closest to $\overrightarrow{\text{un-rich}}$ in terms of *CosSim*: do we actually find $\overrightarrow{\text{poor}}$?

[3] If the vectors are slightly different across pairs (as shown on the previous slide), one can average them to remove noise.

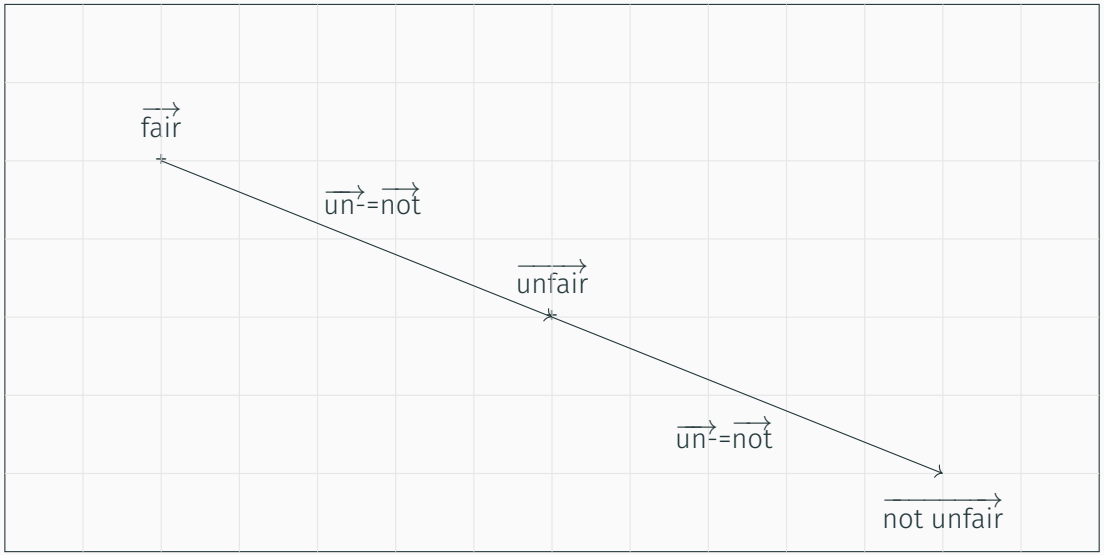
Limits of word embeddings

Lexical ambiguity

- Simple embeddings map words (strings of characters) to vectors.
- What if instead of *poor*, we had looked at a word like *mean*? or *kind*?
- To deal with **lexical ambiguity**, one can:
  - Disambiguate **within the training dataset**, by replacing *mean* by *mean1* under the *nasty* reading, by *mean2* under the *average* reading, by *mean3* under the *intend* meaning etc. But this is extremely tedious!
  - Keep the dataset ambiguous, but derive word vectors based on a word *and its context*. Such models are called **contextual word embeddings** (as opposed to the basic **static embeddings**).
- A related problem is **context-dependence** (*gigantic cockroach* vs. *juvenile/gigantic T. Rex*). We know the difference because we have access to word knowledge. But models learning vectors just based on text may not.

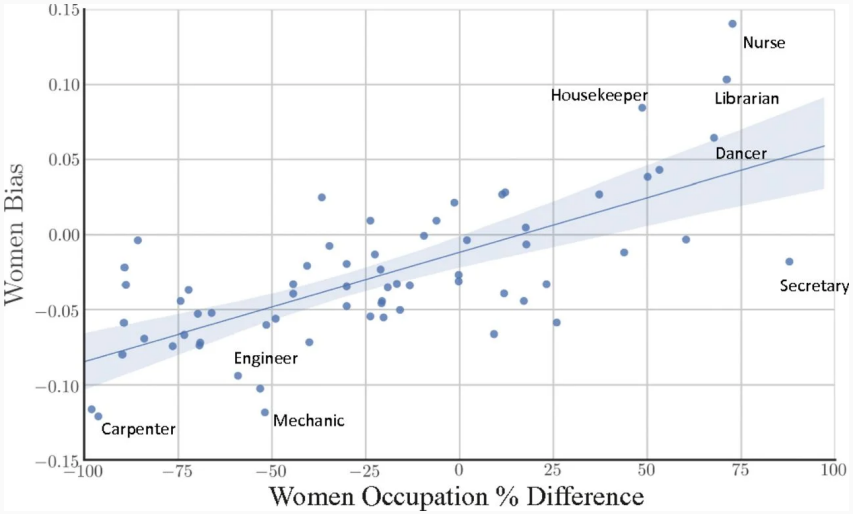
Functional elements

- We have discussed how the relation between two antonymic adjectives like *fair* and *unfair*, was itself a vector ($\overrightarrow{\text{un-}}$) connecting the two adjectives.
- We can also consider the word-vector for *not*. Let's assume for simplicity $\overrightarrow{\text{un-}}=\overrightarrow{\text{not}}$.
- Assuming the vector of two adjacent words is the sum of the word vectors of each word ($\overrightarrow{w_1\ w_2}=\overrightarrow{w_1}+\overrightarrow{w_2}$), where do we predict the vector if *not unfair* to be? Is it closer to $\overrightarrow{\text{fair}}$, or to $\overrightarrow{\text{unfair}}$?



Biases

- *CosSim* measures and the embeddings' topology can incorporate bias.
- This paper studies how embeddings evolve over time and shows how their dynamics correlates with quantifiable changes in US society, e.g. demographic and occupation shifts.
- Embeddings are shown to reflect changes in the descriptions of genders and ethnic groups during the women's movement in the 1960s-1970s and Asian-American population growth in the 1960s and 1980s.



Limits summary

- Word embeddings are great at capturing the meaning of adjectives, nominals, and certain predicates, that intuitively correspond to stable sets of “verifiers”.
- But they have a harder time with **ambiguous** or “**functional**” elements, like negation, but also certain adjectives (subsective, privative, modal), because such elements do not have a fixed meaning in and of themselves, their meaning is their effect on their varied argument(s)!
- Embeddings incorporate **biases** and as such can be used to shed light on societal shifts; but this also means they should also be used with care in downstream tasks!

Beyond word embeddings

- One can map emojis, images, sounds, or entire sentences to vectors too.
- And one can look for interesting regularities in the resulting spaces.

Word2Vec embeddings

Simple embedding models

- The textbook presents two methods to derive word-vectors directly based on **co-occurrences**.
- In both cases, we apply clever transformations to these co-occurrences, to remove spurious information.
- **Word-document co-occurrences** are leveraged by the Term Frequency-Inverse Document Frequency (TF-IDF) method. The word's features are (roughly) the documents of the dataset.
- **Word-word co-occurrences** are leveraged by Pointwise Positive Mutual Information (PPMI). The word's features are (roughly) all the words of the dataset.
- You can read about these relatively simple methods. Their main issue is the **dimensionality**: they produce very big, sparse vectors (with lots of near-zero components). Information is not very well-packaged.
- We'll now briefly go through a more elaborate method, **Word2Vec Skip-gram**.

The Distributional Hypothesis

- Starting in the 50s, linguists (Joos, Firth, Harris...) expressed the idea that the **"meaning" of words** should be determined by the company they keep (their **context** or "distribution").
- For instance, if you're a word that occurs near verbs like *eating*, *cooking*, *baking*, and other food-related nominals or adjectives, then you're probably a word about food!
- This "distributional" approach to meaning incorporates both syntactic and semantic information, as has been argued to underlie language acquisition in children ("**bootstrapping**").[4]

[4] But kids unlike computers also have access to "real" situations! See "cross-situational" learning.

Word2Vec and the distributional hypothesis

- Word2Vec is based on a specific interpretation of the Distributional Hypothesis:

Word2Vec's objective is to make the **vector of any word** and the **vectors of its various contexts similar** to each other.

- This way, words that occur in **similar contexts** will have **similar vectors**.

Representing a word an its context

(3) Raccoons are among the cutest animals around, but the primary carriers of rabies since the nineties. (Cambridge Day)

- Suppose we want to derive a vector for *animals*, $\overrightarrow{\text{animals}}$ (**target** word vector).
- According to the Distributional Hypothesis, *animals* must be close in meaning to its **context**.
- Let's assume the context of *animals* in (3) is the whole sentence, minus *animals*.[5] This context is shown in (3′)

(3′) Raccoons are among the cutest _____ around, but the primary carriers of rabies since the nineties.

[5] In practice, we'd use a fixed-size window centered around the target word *animals*–a bit similar to *n*-gram models.

Word embedding and context embedding

(3′) Raccoons are among the cutest _____ around, but the primary carriers of rabies since the nineties.

- The target $\overrightarrow{\text{animals}}$ being close to (or "making sense" in) its context suggests that for any word w in the context, $\overrightarrow{\text{animals}}$ and $\overrightarrow{w}$ must be relatively close. For instance, $\overrightarrow{\text{animals}}$ and $\overrightarrow{\text{raccoons}}$ should be close.
- Let's assume something slightly more elaborate: instead of having one single mapping from words to vectors $w \mapsto \overrightarrow{w}$, we have 2 mappings:
  - A mapping between words seen as targets and "target vectors": $w \mapsto \overrightarrow{t(w)}$
  - A mapping between words seen as context words and "context vectors": $w \mapsto \overrightarrow{c(w)}$
- Therefore, we'd like $\overrightarrow{\mathbf{t}(\text{animals})}$ and $\overrightarrow{\mathbf{c}(\text{raccoons})}$ to be close, $\overrightarrow{\mathbf{t}(\text{animals})}$ and $\overrightarrow{\mathbf{c}(\text{among})}$ to be close etc.

Measuring closeness in Word2Vec

- To measure closeness between target and context word vectors, we use a variant of the *CosSim* measure: the dot product.

$$\vec{u}.\vec{v} = u_1v_1 + u_2v_2 + ... + u_kv_k, \text{with } \vec{u} = \begin{bmatrix} u_1 \\ u_2 \\ ... \\ u_k \end{bmatrix} \text{ and } \vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ ... \\ v_k \end{bmatrix}$$

- The dot product is sensitive to the vector magnitudes, and is unbounded: a very negative dot product indicates a negative association, a very positive dot product a positive association and a zero dot product no association between the vectors.

Dot product practice

$$\overrightarrow{\text{t(animals)}} = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}, \overrightarrow{\text{c(raccoons)}} = \begin{bmatrix} 0 \\ -1 \\ 5 \end{bmatrix}, \overrightarrow{\text{c(the)}} = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$$

- What's $\overrightarrow{\text{t(animals)}} \cdot \overrightarrow{\text{c(raccoons)}}$? $1 \times 0 + 0 \times -1 + 2 \times 5 = 10$
- What's $\overrightarrow{\text{t(animals)}} \cdot \overrightarrow{\text{c(the)}}$? $1 \times 1 + 0 \times 1 + 2 \times -1 = -1$
- $\overrightarrow{\text{t(animals)}}$ and $\overrightarrow{\text{c(raccoons)}}$ are "hot" in the same component (the third one); their dot product is relatively high. $\overrightarrow{\text{t(animals)}}$ and $\overrightarrow{\text{c(the)}}$ are not "hot" in the same components; their dot product is lower.

Spelling out target-context similarity

- For every word w in the context of *animals*, we want $\overrightarrow{c(w)} \cdot \overrightarrow{t(animals)}$ to be fairly high.
- We can "squish" this quantity to belong to $]0;1[$, thanks to the sigmoid function: $\sigma(\overrightarrow{c(w)} \cdot \overrightarrow{t(animals)})$ corresponds to the probability that *animals* and a context word w "make sense" together.
- This can be done for any w in the context of *animals*. We can then compute the product of all these sigmoids/probabilities:

$$P(animals \text{ makes sense in } (3')) = \prod_{w \in (3')} \sigma(\overrightarrow{t(animals)}.\overrightarrow{c(w)})$$

- We want this quantity to be as close to 1 as possible.

Negative sampling

- Suppose we are training our model only on "real" sentences that make sense. For any sentence S and w in S, we thus want $P(w \text{ makes sense in } S \setminus w)$ to be close to 1.
- This is easy to achieve! Just assign all words the same vectors, with fairly high components, e.g. 10 everywhere. This way, for any two words w and w', $\vec{w}.\vec{w'}$ will be high, $\sigma(\vec{w}.\vec{w'})$ will be close to 1, and any product of such σ's will be close to 1 too! But then our word vectors are meaningless.
- To prevent this, we also train our model on **negative** examples, i.e. sentences S^- that do not make sense. In that case, we'll want $P(w \text{ makes sense in } S^- \setminus w)$ to be close to 0.

Negative sample example

- I made (4) by replacing every word in (3) but *animals* by a "crazy" word. (4′) represents the context of *animals* in (4).

(4) Juice little frenzy helicopter shine animals slurp, game eloquent mud painter these cleverly from law brisket.

(4′) Juice little frenzy helicopter shine _____ slurp, game eloquent mud painter these cleverly from law brisket.

$$P(\textit{animals} \text{ makes sense in } (4')) = \prod_{w \in (4')} \sigma(\overrightarrow{t(\textit{animals})}.\overrightarrow{c(w)})$$

- For any word w in (4′), we want the dot product $\overrightarrow{\text{t(animals)}} \,.\, \overrightarrow{\text{c(w)}}$ to be very negative.
  - Then, $\sigma(\overrightarrow{t(\textit{animals})}.\overrightarrow{c(w)})$ will be close to 0.
  - And $P(\textit{animals}$ makes sense in $(4'))$ will be close to 0 too.
- This could not be achieved if all word-vectors were assigned high components (optimal solution with "real" samples only).

Extra: learning in Word2Vec

Objective and learning procedure

- "Positive", "real" sentences make us learn which words should be close to each other.
- "Negative", "crazy" sentences make us learn which words should *not* be close to each other.
- Let's assume each target word appears in one positive sentence S^+ and one negative sentence S^-. We want the following product to be close to 1:

$$\underbrace{P(w \text{ makes sense in } S^+)}_{\text{ideally 1}}\underbrace{(1 - \underbrace{P(w \text{ makes sense in } S^-)}_{\text{ideally 0}})}_{\text{ideally 1}}$$

$$= \prod_{w' \in S^+} \sigma(\overrightarrow{t(w)}.\overrightarrow{c(w')}) \times \left(1 - \prod_{w' \in S^-} \sigma(\overrightarrow{t(w)}.\overrightarrow{c(w')})\right)$$

- In practice we -log this quantity to have something that lives in $[0; +\infty[$ and should be minimized.

Learning t and c

$$L = -log\left(\prod_{w' \in S^+} \sigma(\overrightarrow{t(w)}.\overrightarrow{c(w')}) \times \left(1 - \prod_{w' \in S^-} \sigma(\overrightarrow{t(w)}.\overrightarrow{c(w')})\right)\right)$$

- We want to **minimize the loss** L.
- L depends on the target and the context word vectors!
- We can use gradient descent to update $\overrightarrow{t(w)}$ and $\overrightarrow{c(w')}$ for all $w' \in S^+ \cup S^-$ in a way that decreases this function, i.e. brings target and context representations closer to each other in the case of the positive sample S^+, and further away from each other in the case of the negative sample S^-.
- We can use t as the output embedding, or some function of t and c, e.g. their sum.

Lab

https://colab.research.google.com/drive/1fOVtddG-bQHTEZR6w1lTKaRLVoAMBQbhK?usp=sharing